



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

LEARN DEEP: PLACEMENT PREPARATION

Submitted by

RAKESH R (230701257)

in partial fulfillment for the course

CS23621 – MOBILE APPLICATION DEVELOPMENT LABORATORY

of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

MAY 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI-602105

BONAFIDE CERTIFICATE

Certified that this project report titled “**LEARN DEEP: PLACEMENT PREPARATION**” is the bonafide work of **RAKESH R (230701257)** who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Submitted for the End semester practical examination to be held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

Initially, we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our chairman **Mr. S. MEGANATHAN, B.E, F.I.E.,** our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E, M.S.,** for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. E.M MALATHY, M.E., Ph.D.,** Professor and Head of the Department of Computer Science and Engineering and our mentor **Mrs B.Deepa M.E., (Phd).,** Department of Computer Science and Engineering for his useful tips during our review to build our project.

RAKESH R (230701257)

ABSTRACT

The Interview Prep Tracker is a feature-rich Android mobile application developed using Kotlin and Jetpack Compose, designed to provide engineering students and job seekers with a comprehensive, structured, and data-driven platform for interview preparation. In an increasingly competitive technology recruitment landscape, candidates require a systematic approach that combines skill self-assessment, targeted knowledge acquisition, progress analytics, and role-specific guidance. The Interview Prep Tracker addresses this need through an integrated suite of modules built upon a robust technical architecture.

The application employs the Model-View-ViewModel (MVVM) architectural pattern, ensuring a clean separation between business logic, data management, and the user interface layer. Firebase Authentication is used to provide secure user registration and login, with real-time session management through Kotlin coroutines and the Firebase Authentication State Listener API. Firebase Cloud Firestore serves as the primary cloud data store, maintaining per-user collections for skills and quiz results with real-time synchronisation capabilities facilitated through Kotlin Flow and the callbackFlow builder.

The Skills Tracking module allows users to create, update, and delete professional skills, each annotated with a proficiency level spanning Beginner to Expert, persisted in Firestore under a hierarchical user-document structure. The Quiz Module provides Multiple Choice Questions across nine technical and aptitude topics—Kotlin, Java, Python, DBMS, Object-Oriented Programming, Operating Systems, Computer Networks, Aptitude, and Data Structures and Algorithms—at three difficulty levels (Easy, Medium, Hard), with a per-question thirty-second countdown timer, immediate answer revelation with explanations, and automatic result persistence to Firestore.

The Role Recommendation Engine implements a weighted scoring algorithm that evaluates the user's registered skill set against predefined profiles for eight technology roles, producing a ranked list of role matches with percentage scores, match justifications, and skill gap analyses. The Roadmap Module leverages the highest-scoring recommended role to render a structured, expandable preparation timeline with estimated week-by-week milestones and curated learning resources. The Dashboard aggregates quiz performance analytics including average scores, per-topic and per-

difficulty breakdowns, quiz streak computation, and recent result history, rendered using the Vico charting library integrated with Jetpack Compose.

Dependency injection is managed through Hilt (Dagger-Hilt), with Firebase singleton instances provisioned via an AppModule scoped to the application lifecycle. Local preferences including dark mode state, quiz streak, and last quiz date are persisted using Jetpack DataStore Preferences. Navigation is implemented using Navigation Compose with type-safe route arguments and animated transitions. The Material Design 3 theming system provides both light and dark theme support with dynamic user toggle capability.

The application was tested through structured functional test cases covering authentication flows, CRUD operations on skills, quiz session mechanics, recommendation accuracy, and Firestore synchronisation reliability. The project demonstrates the practical application of modern Android development practices and serves as a complete reference implementation of the MVVM pattern with Firebase integration.

Keywords: *Android, Kotlin, Jetpack Compose, MVVM, Firebase Authentication, Firestore, Hilt, Navigation Compose, Material Design 3, Quiz Module, Role Recommendation, DataStore*

TABLE OF CONTENTS

Content	Page
---------	------

Bonafide Certificate	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vi
List of Abbreviations	vii
Chapter 1: Introduction	1
1.1 Overview	1
1.2 Motivation	2
1.3 Objectives	3
Chapter 2: System Design and Architecture	4
2.1 System Architecture	4
2.2 Module Description	5
2.3 Data Models	7
2.4 Firebase Structure	8
2.5 MVVM Architecture	9
Chapter 3: Implementation	11
3.1 Technology Stack	11
3.2 Firebase Integration	12
3.3 Authentication Flow	13
3.4 Dashboard Module	14
3.5 Skill Tracking Module	15
3.6 Roadmap Module	16
3.7 Quiz Module	17
3.8 Recommendation Engine	18
3.9 Navigation Flow	19
3.10 Firestore Structure	20
3.11 UI Design	21
Chapter 4: Testing	22
4.1 Testing Strategy	22
4.2 Functional Testing	22
4.3 UI Testing	23

4.4 Firebase Testing	24
4.5 Test Cases Table	24
Chapter 5: Results and Discussion	28
5.1 Functional Results	28
5.2 Performance Results	29
5.3 UI/UX Results	29
5.4 Analytics Features	30
5.5 Firebase Synchronisation Results	30
Chapter 6: Conclusion and Future Work	31
6.1 Conclusion	31
6.2 Future Enhancements	32
References	33

LIST OF FIGURES

Figures	page
Figure 2.1 — Overall System Architecture Diagram	13
Figure 3.1 — Login and Signup screen	21
Figure 3.2 — Dashboard UI	23
Figure 3.3 — Skills Selection Screen	25
Figure 3.4 — Learning Roadmap based on desired roles	27
Figure 3.5 — Quiz center screen	29
Figure 3.6 — Role insights based on Skills screen	30

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DI	Dependency Injection
DSA	Data Structures and Algorithms
DBMS	Database Management System
DXA	Device-independent XML units (typographic measurement)
FCM	Firebase Cloud Messaging
MVVM	Model–View–ViewModel
MCQ	Multiple Choice Question
OOP / OOPs	Object-Oriented Programming
OS	Operating System
SDK	Software Development Kit
UI	User Interface
UX	User Experience
UID	Unique Identifier
ViewModel	View-Model component in MVVM
XML	eXtensible Markup Language
IDE	Integrated Development Environment
CN	Computer Networks
KSP	Kotlin Symbol Processing

CHAPTER 1

INTRODUCTION

1.1 Overview

The Interview Prep Tracker is a comprehensive Android application conceived and developed to provide engineering students and technology professionals with an integrated, mobile-first platform for structured interview preparation. The application consolidates multiple dimensions of interview readiness—technical skill tracking, topic-specific multiple choice quizzes, role-based career roadmaps, and AI-style role recommendation—into a single, cohesive user experience powered by cloud synchronisation and modern reactive programming paradigms.

In the current technology recruitment landscape, candidates are expected to demonstrate proficiency across a broad spectrum of subjects including programming languages, data structures and algorithms, database management systems, operating systems, computer networks, and behavioural aptitude. Preparing for interviews across these domains demands discipline, systematic tracking, and accurate self-assessment. Generic study resources fail to offer the personalisation and progress measurement that candidates require. The Interview Prep Tracker addresses this gap by combining data-driven analytics with a curated quiz engine and contextual roadmap guidance.

The application is developed entirely in Kotlin for the Android platform, leveraging Jetpack Compose as the declarative user interface framework and following the Model-View-ViewModel (MVVM) architectural pattern for maintainability and testability. Firebase Authentication and Firebase Cloud Firestore are employed as the backend services for identity management and cloud data persistence respectively. Dependency injection is managed through Hilt (Dagger-Hilt) to ensure a clean, modular, and testable codebase. Navigation between screens is handled by the Navigation Compose library with animated transitions and type-safe route parameters.

The application package identifier is `com.interviewprep.tracker` and the project targets Android API Level 26 (Android 8.0 Oreo) as the minimum SDK, compiled against API Level 35. It comprises six primary functional modules: Authentication, Dashboard, Skill Tracking, Quiz, Roadmap, and Role Recommendation. Each module is backed by corresponding ViewModel classes, repository classes, and Jetpack Compose screen composables, following a strict MVVM separation of concerns.

1.2 Motivation

The motivation for developing the Interview Prep Tracker stems from the observation that interview preparation, despite being a critical phase in a software engineering career, remains largely fragmented and unstructured. Students often rely on a disparate collection of tools—flashcard applications for vocabulary, LeetCode for coding problems, YouTube playlists for concept revision, and spreadsheet trackers for skill inventories. This fragmentation leads to inefficiency, duplicated effort, and an absence of measurable progress tracking.

Existing preparation platforms are predominantly web-based, lack offline capability, and do not adapt their content or guidance to the specific skill profile of the individual user. They treat all users as homogeneous candidates regardless of whether the user is targeting an Android development role, a data science position, or a full-stack engineering role. The absence of personalisation means that users spend significant time on content that may be irrelevant to their career aspirations.

Furthermore, the absence of a daily habit-forming mechanism such as a quiz streak and performance trend visualisation means that users lose motivation over extended preparation periods. Research in educational technology consistently demonstrates that spaced repetition, timely feedback, and visual progress indicators significantly improve learning outcomes and retention. These insights motivated the inclusion of a per-question countdown timer, immediate post-answer explanation, streak tracking, and the dashboard analytics module in this application.

The proliferation of Android smartphones in the Indian subcontinent, with hundreds of millions of active users across economic demographics, presents a compelling case for a mobile-first solution. Android applications offer ubiquitous access, push notification capability, and offline data availability through local caching mechanisms such as Jetpack DataStore, making them inherently better suited for daily study habit formation compared to web applications requiring active browser sessions.

The choice of Firebase as the backend platform is motivated by its serverless architecture, real-time data synchronisation capabilities, and the elimination of server provisioning and maintenance overhead. Firebase Firestore's document-collection hierarchy maps naturally to the per-user data model required by the application, while Firebase Authentication provides enterprise-grade identity management with minimal configuration.

1.3 Objectives

The primary objectives of the Interview Prep Tracker project are enumerated as follows:

- To design and implement a secure, cloud-backed Android application for end-to-end interview preparation lifecycle management.
- To provide a structured skill inventory system enabling users to record technical skills with proficiency levels (Beginner, Intermediate, Advanced, Expert) and track skill growth over time through Firebase Firestore with real-time synchronisation.
- To implement a multiple choice quiz engine covering nine technical and aptitude domains with three configurable difficulty levels, a per-question timer, immediate explanatory feedback, and automatic result persistence.
- To develop a role recommendation engine that analyses the user's registered skill set against predefined role profiles and produces ranked recommendations with match percentages, match rationale, and skill gap identification.
- To deliver interactive, role-specific preparation roadmaps with week-by-week milestones, expandable step details, and curated learning resource links, dynamically aligned to the top recommended role.
- To build a real-time analytics dashboard presenting quiz performance trends, per-topic and per-difficulty score breakdowns, consecutive day quiz streak computation, and top role recommendation summary.
- To implement a scalable MVVM architecture with Hilt-based dependency injection, ensuring clear separation of concerns, testability, and long-term maintainability of the codebase.
- To support dark and light theme preferences persisted across application sessions using Jetpack DataStore Preferences, and to deliver a polished Material Design 3 user interface with animated transitions and responsive layouts.

CHAPTER 2

SYSTEM DESIGN AND ARCHITECTURE

2.1 System Architecture

The Interview Prep Tracker is architected as a single-module Android application following the Model-View-ViewModel (MVVM) pattern recommended by Google for modern Android development. The overall architecture is stratified into three principal layers: the Presentation Layer, the Domain/ViewModel Layer, and the Data Layer. Each layer has well-defined responsibilities and communicates with adjacent layers through reactive streams using Kotlin StateFlow and Flow, ensuring unidirectional data flow and preventing tight coupling between UI components and business logic.

The Presentation Layer consists exclusively of Jetpack Compose composable functions organised by feature screen: LoginScreen, RegisterScreen, DashboardScreen, SkillsScreen, QuizSelectionScreen, QuizScreen, QuizResultScreen, RoadmapScreen, and RecommendationScreen. Composables observe immutable UI state objects exposed by their respective ViewModels through collectAsStateWithLifecycle, ensuring lifecycle-aware state collection that automatically cancels collection on Composition disposal. Reusable UI primitives such as StatCard, AnimatedProgressBar, SectionHeader, and ProficiencyChip are extracted into a dedicated CommonComponents file for cross-feature reusability.

The ViewModel Layer contains six HiltViewModel-annotated ViewModel classes—AuthViewModel, DashboardViewModel, QuizViewModel, SkillsViewModel, RecommendationViewModel, and RoadmapViewModel—each injected with its required repository dependencies through Hilt's constructor injection. ViewModels expose reactive StateFlow properties encapsulating screen-specific state objects and execute business logic within viewModelScope, using Kotlin coroutines for all asynchronous operations. No Android framework classes are imported into ViewModels, maintaining strict testability.

INTERVIEW PREP TRACKER – ARCHITECTURE DIAGRAM

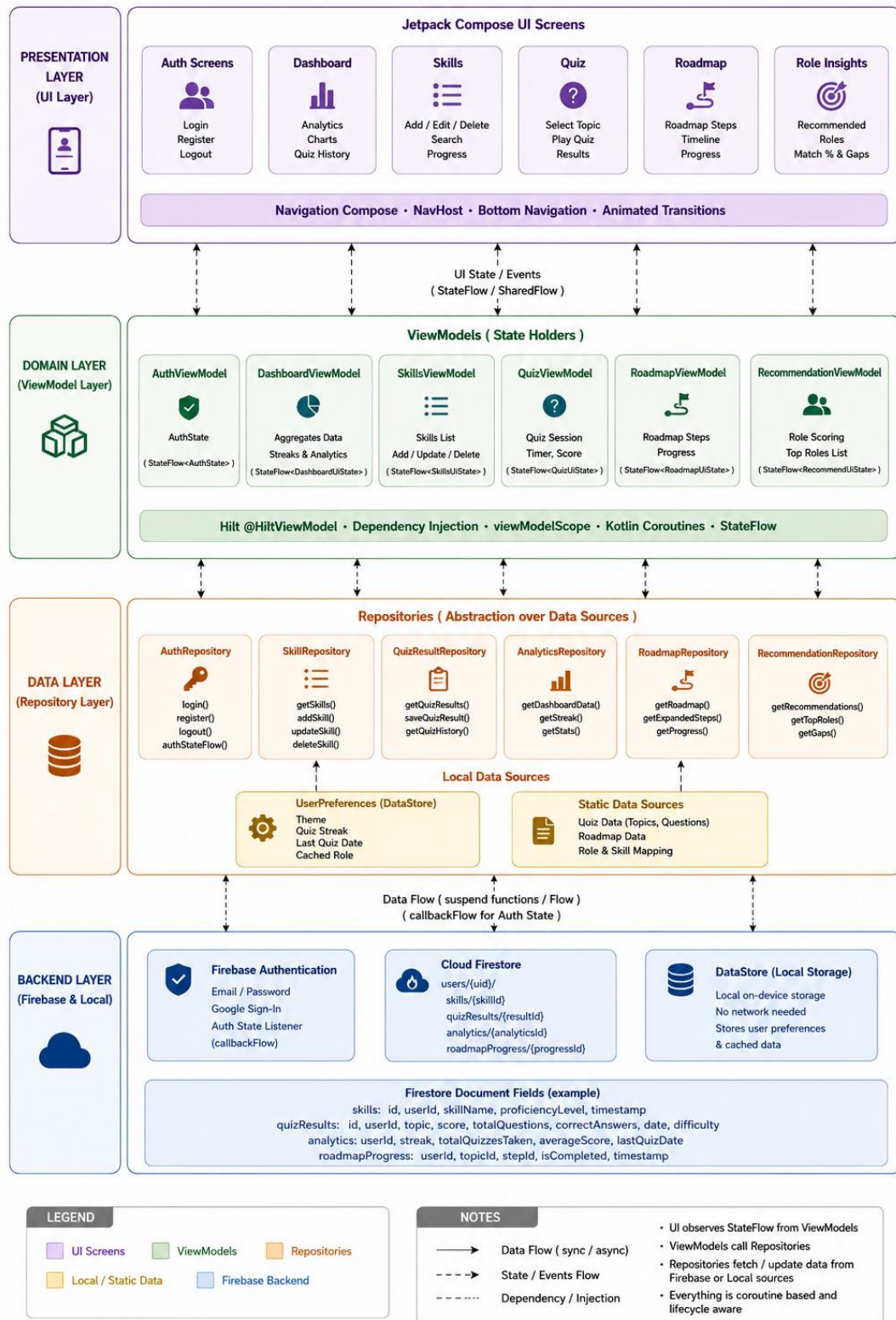


Figure 2.1 : Overall System Architecture Diagram

The Data Layer is further subdivided into remote repositories (AuthRepository, SkillRepository, QuizResultRepository) managing Firebase interactions, a local repository (UserPreferencesRepository) managing DataStore Preferences, and predefined data objects (QuizData, RoadmapData, RoleMappings) encapsulating static application content. Remote repositories are annotated with `@Singleton` and injected through Hilt, receiving FirebaseAuth and FirebaseFirestore instances provided by the AppModule DI module. This ensures a single, application-scoped instance of each Firebase service throughout the application lifecycle.

Figure 2.1 illustrates the layered system architecture. At the top reside the Compose UI screens receiving user events. These communicate with ViewModel classes through event handlers (onClick callbacks, etc.). ViewModels coordinate with repositories to execute data operations and emit updated state via StateFlow. Repositories interact with Firebase services (Auth, Firestore) and local DataStore. All navigation is orchestrated by NavGraph.kt using the Navigation Compose NavHost, with the AppBottomNavBar providing five-tab primary navigation for authenticated screens.

2.2 Module Description

The application is logically partitioned into seven functional modules, each encapsulating a distinct domain of functionality. The following table provides a structured description of each module.

Module	Description	Key Components
Authentication Module	Manages user registration, login, and sign-out via Firebase Authentication with real-time auth state monitoring.	AuthViewModel, AuthRepository, LoginScreen, RegisterScreen
Dashboard Module	Aggregates and presents quiz analytics, skill counts, streak, top recommendation, and recent quiz history in a pull-to-refresh layout with Vico charts.	DashboardViewModel, DashboardData, DashboardScreen, StatCard, AnimatedProgressBar
Skill Tracking Module	Provides full CRUD operations on user skills with proficiency levels. Supports search filtering, swipe-to-delete gesture, and inline edit dialog.	SkillsViewModel, SkillRepository, SkillsScreen, ProficiencyLevel enum
Quiz Module	Presents topic/difficulty selection, conducts MCQ quiz sessions with a 30-second countdown timer per question, reveals answers	QuizViewModel, QuizResultRepository, QuizData,

Module	Description	Key Components
	with explanations, and persists results to Firestore.	QuizSelectionScreen, QuizScreen, QuizResultScreen
Roadmap Module	Renders expandable week-by-week preparation roadmaps for eight technology roles, dynamically loading the top recommended role on initialisation.	RoadmapViewModel, RoadmapData, RoadmapScreen
Role Recommendation Module	Computes weighted match percentages between user skills and eight role profiles. Presents ranked recommendations with reasons and skill gaps.	RecommendationViewModel, RoleMappings, RecommendationScreen
Navigation Module	Defines the full application route graph with authentication guard, animated transitions, parameterised routes for quiz topic/difficulty/score, and a five-item bottom navigation bar.	NavGraph.kt, Screen sealed class, AppNavGraph, AppBottomNavBar

2.3 Data Models

The application defines its domain entities in a centralised `Models.kt` file within the `com.interviewprep.tracker.model` package. The data model is designed to align with Firebase Firestore's document-collection storage paradigm. The following table enumerates the primary data classes and their fields.

Model Class	Fields	Purpose
User	uid: String, email: String, displayName: String, createdAt: Timestamp	Represents an authenticated application user.
Skill	id: String, userId: String, skillName: String, proficiencyLevel: String, timestamp: Timestamp	Represents a single user skill entry with proficiency and creation timestamp.
QuizResult	id: String, userId: String, topic: String, difficulty: String, score: Int, totalQuestions: Int, percentage: Int, timestamp: Timestamp	Stores the outcome of a completed quiz session in Firestore.
QuizSession	topic: QuizTopic, difficulty: QuizDifficulty, questions: List<QuizQuestion>, answers: MutableMap<Int,Int>, currentIndex: Int, isCompleted: Boolean	Holds the in-memory state of an active quiz session including answer map and current position.

Model Class	Fields	Purpose
QuizQuestion	id: Int, question: String, options: List<String>, correctAnswerIndex: Int, explanation: String, difficulty: QuizDifficulty, topic: QuizTopic	Represents a single MCQ with answer options, correct answer index, and explanation.
RoleRecommendation	roleName: String, matchPercentage: Int, matchReasons: List<String>, missingSkills: List<String>, icon: String, color: Long	Encapsulates a computed role recommendation with scoring details.
RoadmapStep	id: Int, title: String, description: String, resources: List<String>, estimatedWeeks: Int, isCompleted: Boolean	Represents a single step in a preparation roadmap.
Roadmap	role: String, description: String, totalWeeks: Int, steps: List<RoadmapStep>	Groups multiple RoadmapStep entries under a named role.
DashboardData	skills, recentResults, topRecommendation, averageScore, totalQuizzes, streak, topicBreakdown, difficultyBreakdown, isLoading, error	Aggregated UI state object for the Dashboard screen.

The application also defines sealed classes `UiState<T>` (Loading, Success, Error, Empty) and `AuthState` (Loading, Authenticated, Unauthenticated, Error) to model asynchronous operation states across all ViewModels, providing exhaustive when-expression handling in composables.

Enumeration types `ProficiencyLevel` (BEGINNER, INTERMEDIATE, ADVANCED, EXPERT), `QuizDifficulty` (EASY, MEDIUM, HARD), and `QuizTopic` (KOTLIN, JAVA, PYTHON, DBMS, OOPS, OS, CN, APTITUDE, DSA) provide type-safe categorisation throughout the application. Each `ProficiencyLevel` carries a human-readable label and a numeric value enabling ordered comparison.

2.4 Firebase Structure

Firebase Cloud Firestore is structured hierarchically using a root collection named 'users', where each document corresponds to a unique authenticated user identified by their Firebase Authentication UID. Under each user document, two sub-collections exist: 'skills' and 'quizResults'. This structure enforces data isolation at the user level and enables Firestore Security Rules to restrict read/write access to each user's own data exclusively.

Collection Path	Document ID	Fields Stored	Description
users/{uid}/skills	Auto-generated	id, userId, skillName, proficiencyLevel, timestamp	Stores each skill entry added by the user.
users/{uid}/quizResults	Auto-generated	id, userId, topic, difficulty, score, totalQuestions, percentage, timestamp	Stores each completed quiz result.

Both sub-collections are ordered by the 'timestamp' field in descending order for retrieval, ensuring the most recent entries appear first in the UI. The SkillRepository exposes both a real-time Flow-based listener (getSkillsFlow) using addSnapshotListener for live UI updates and a one-time fetch method (getSkillsOnce) for cases where a current snapshot suffices. Similarly, QuizResultRepository provides getResultsFlow for real-time streaming and getResultsOnce with a limit of 50 documents for bounded queries.

Auto-generated document references are obtained via the Firestore .document() method before data is written, allowing the generated ID to be stored as a field within the document itself (the 'id' field). This design pattern eliminates the need for a secondary read operation to obtain the document ID after creation and simplifies document-level operations such as update and delete.

Firestore interactions are performed using Kotlin coroutines with the kotlinx-coroutines-play-services library, which provides the await() extension function on Firebase Tasks, converting callback-based Task APIs into suspend function calls compatible with coroutine scopes.

2.5 MVVM Architecture

The Model-View-ViewModel (MVVM) architectural pattern is the foundational design paradigm of the Interview Prep Tracker. MVVM provides a clean separation of concerns that is particularly well-suited to the reactive, event-driven nature of Android UI development with Jetpack Compose. In this architecture, the View (Compose screens) is responsible solely for rendering UI and dispatching user events; the ViewModel manages UI state and orchestrates data operations; and the Model (repositories and data classes) abstracts all data access logic.

Each ViewModel class in the application is annotated with @HiltViewModel, enabling Hilt to manage its lifecycle and inject its dependencies. ViewModel instances survive configuration changes (such as screen rotation) because they are scoped to the NavBackStackEntry or Activity lifecycle as managed

by `hiltViewModel()` or `viewModel()` composable functions. This ensures that in-progress quiz sessions, loaded skills lists, and dashboard analytics are not discarded on configuration change.

State is exposed from ViewModels to composables exclusively through Kotlin `StateFlow` and `Flow`, which are lifecycle-safe observable streams. In composables, state is collected using `collectAsStateWithLifecycle()`, which automatically pauses collection when the app is backgrounded and resumes when foregrounded, preventing unnecessary background computation. Unidirectional data flow is maintained: events flow from the UI up to the ViewModel, and state flows down from the ViewModel to the UI.

The `DashboardViewModel` demonstrates the power of reactive composition by combining three independent data streams—`getSkillsFlow` (skills list), `getResultsFlow` (quiz results), and `quizStreak` (`DataStore` preference)—using the Kotlin `Flow` combine operator. The combined stream recomputes the complete `DashboardData` object whenever any of the constituent streams emits a new value, ensuring the dashboard always reflects the latest state across all data sources without manual refresh logic.

Figure 2.2 illustrates the MVVM layer diagram. The Compose UI layer observes `StateFlow<UiState<T>>` from the ViewModel layer. Events from the UI trigger ViewModel functions such as `addSkill()`, `selectAnswer()`, or `loadRecommendations()`. The ViewModel delegates to repository methods (suspend functions or `Flow`-returning functions) and updates the exposed `StateFlow`. Repositories communicate with Firebase services or `DataStore`, emitting results back through the coroutine suspension mechanism.

CHAPTER 3

IMPLEMENTATION

3.1 Technology Stack

The Interview Prep Tracker is implemented using a curated selection of modern Android development technologies. The following table provides a complete enumeration of the technology stack with corresponding versions and functional roles within the application.

Technology	Version	Role in Application
Kotlin	2.x (JVM 17)	Primary programming language for all application code
Jetpack Compose	BOM 2024.x	Declarative UI framework for all screen composables
Navigation Compose	Latest stable	In-app navigation, route management, and back stack control
Material Design 3	Latest stable	UI theming, typography, colour schemes, and component library
Hilt (Dagger-Hilt)	Latest stable	Dependency injection framework, scoped instance management
Kotlin Symbol Processing (KSP)	Latest stable	Annotation processing for Hilt code generation
Firebase Authentication	BOM-managed	User registration, login, sign-out, and auth state monitoring
Firebase Cloud Firestore	BOM-managed	Cloud NoSQL database for skills and quiz results persistence
Kotlin Coroutines	1.x	Asynchronous programming, suspend functions, Flow, StateFlow
kotlinx-coroutines-play-services	1.x	await() extension for Firebase Task APIs
Jetpack DataStore Preferences	Latest stable	Local persistent storage for dark mode, streak, and preferences
Vico (vico-compose, vico-compose-m3, vico-core)	Latest stable	Chart rendering for dashboard analytics visualisations
AndroidX Lifecycle ViewModel Compose	Latest stable	hiltViewModel() and collectAsStateWithLifecycle()

Technology	Version	Role in Application
AndroidX SplashScreen	Latest stable	Launch splash screen with auth-state-conditioned delay
Coil Compose	Latest stable	Asynchronous image loading in composables
Android Gradle Plugin	8.x	Build system and APK packaging
Gradle Kotlin DSL	8.9	Build script configuration in Kotlin (.kts files)

The project uses a version catalog (libs.versions.toml) managed by Gradle's version catalog feature, ensuring consistent dependency versioning across the build. The Firebase Bill of Materials (BOM) is used to align all Firebase library versions, eliminating manual version coordination. The project compiles against Android compileSdk 35 with a minimum SDK of 26, targeting API 35 for the release build, with ProGuard minification enabled in release configuration.

3.2 Firebase Integration

Firebase integration in the Interview Prep Tracker is implemented through the AppModule Hilt dependency injection module, which resides in the di package and is annotated with `@Module`, `@InstallIn(SingletonComponent::class)`. This module provides two application-scoped singleton instances: `FirebaseAuth.getInstance()` and `FirebaseFirestore.getInstance()`. By provisioning these instances through Hilt, the application ensures that all repository classes share the same Firebase connection objects without requiring manual singleton management or global static references.

The `google-services.json` configuration file, generated from the Firebase Console upon project registration, contains the application's Firebase project credentials including the API key, project ID, storage bucket, and messaging sender ID. This file is placed in the `app/` module directory and processed by the Google Services Gradle plugin (`com.google.gms.google-services`) during the build process, automatically configuring the `FirebaseApp` instance before the `Application` class executes.

Firebase Authentication is accessed through the `AuthRepository` class, which uses the `FirebaseAuth.AuthStateListener` callback wrapped in a `callbackFlow` builder to produce a continuous `Flow<AuthState>` stream. This stream emits `AuthState.Authenticated` when a non-null `currentUser` is present and `AuthState.Unauthenticated` otherwise, providing a reactive, real-time signal of authentication status that the `AuthViewModel` transforms into a `StateFlow<AuthState>` using `stateIn`

with `SharingStarted.WhileSubscribed(5000)`. The 5000ms timeout ensures the `StateFlow` remains active during short-lived configuration changes.

Firebase Cloud Firestore is accessed through `SkillRepository` and `QuizResultRepository`, both of which use the `addSnapshotListener` API wrapped in `callbackFlow` to produce real-time streaming data. The snapshot listeners are registered upon `Flow` collection and removed in the `awaitClose` block, preventing listener leaks when the collecting coroutine is cancelled. All write operations (add, update, delete) are implemented as suspend functions using the `kotlinx-coroutines-play-services` `await()` extension and return `Kotlin Result<T>` types for explicit success/failure handling in the `ViewModel` layer.

3.3 Authentication Flow

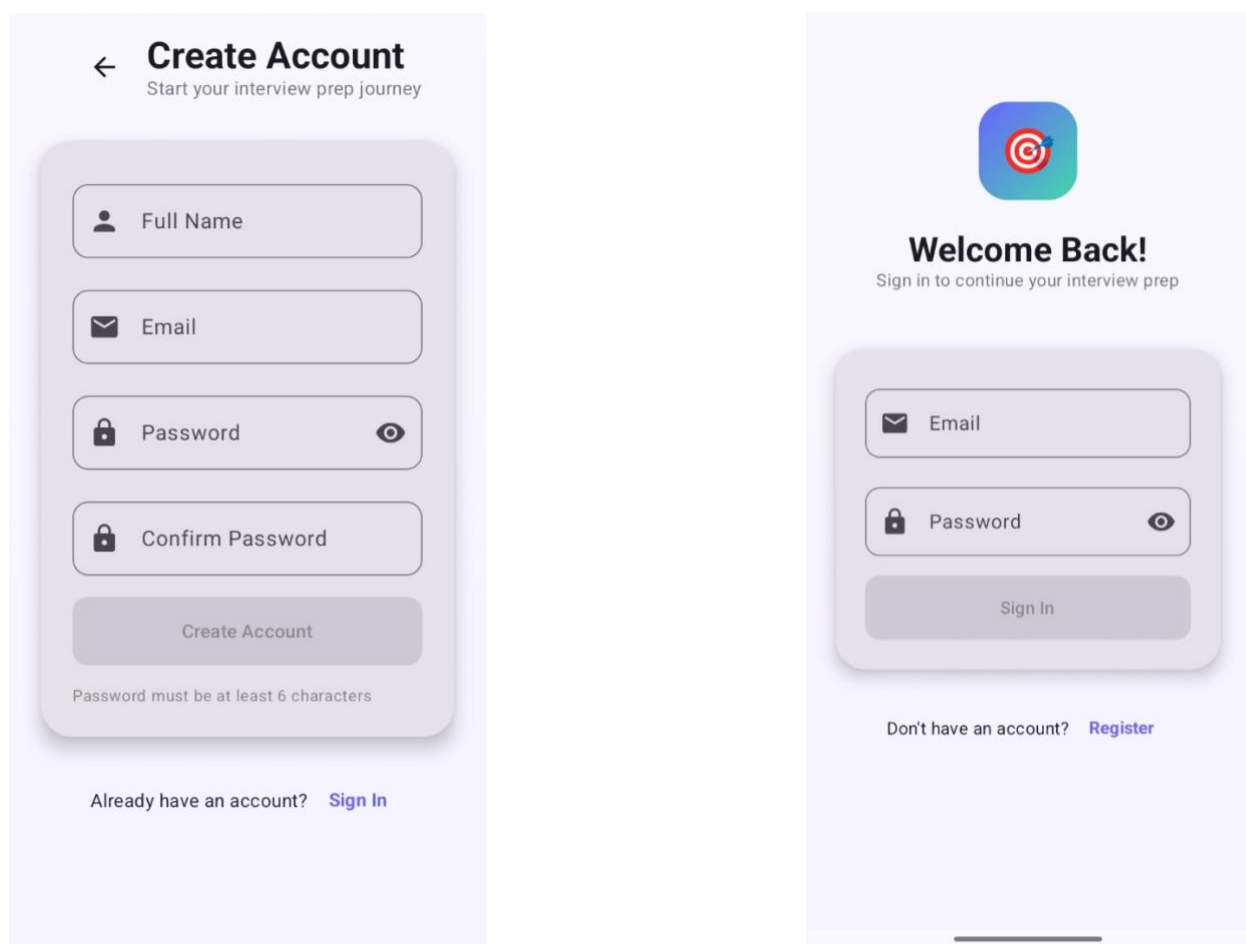


Figure 3.1 : Login and Signup screen

The authentication flow begins at application launch in MainActivity, where the SplashScreen API is used to display a splash screen that remains visible until the authentication state resolves from AuthState.Loading to either Authenticated or Unauthenticated. The AuthViewModel's authState StateFlow, derived from the Firebase AuthStateListener, drives this determination. Once resolved, the NavGraph's startDestination is set to Screen.Dashboard.route for authenticated users or Screen.Login.route for unauthenticated users.

The LoginScreen composable collects the authState, loginError, and isLoading StateFlows from AuthViewModel. Upon the user submitting their email and password, the composable invokes viewModel.signIn(), which delegates to AuthRepository.signIn(), calling firebaseAuth.signInWithEmailAndPassword(email, password).await(). A successful result triggers the AuthStateListener to emit AuthState.Authenticated, which propagates through the StateFlow and causes the NavGraph to navigate to the Dashboard while clearing the Login route from the back stack using popUpTo(Screen.Login.route) { inclusive = true }.

The RegisterScreen follows an identical pattern, invoking viewModel.register() which calls AuthRepository.register(), creating a new user via firebaseAuth.createUserWithEmailAndPassword(). The displayName parameter is stored on the User domain object returned to the ViewModel. Registration errors such as email already in use or weak password are communicated through the registerError StateFlow and displayed as a Snackbar or error text on the screen.

Sign-out is initiated from the Dashboard's top app bar overflow menu, which calls viewModel.signOut(). AuthViewModel.signOut() invokes both authRepository.signOut() (which calls firebaseAuth.signOut()) and prefsRepository.clearAll() (which clears all DataStore entries, including cached role and streak data). The AuthStateListener then emits AuthState.Unauthenticated, causing the NavGraph to navigate to Screen.Login.route with popUpTo(0) { inclusive = true }, clearing the entire back stack.

3.4 Dashboard Module

The DashboardScreen serves as the application's primary home screen, providing a holistic snapshot of the user's interview preparation progress. It is implemented as a Scaffold with a TopAppBar featuring the application title, a theme toggle button, and a sign-out overflow menu. The content area

is wrapped in a PullToRefreshBox (Material 3 Experimental API), enabling manual data refresh through a pull gesture that re-invokes `viewModel.loadDashboard()`.

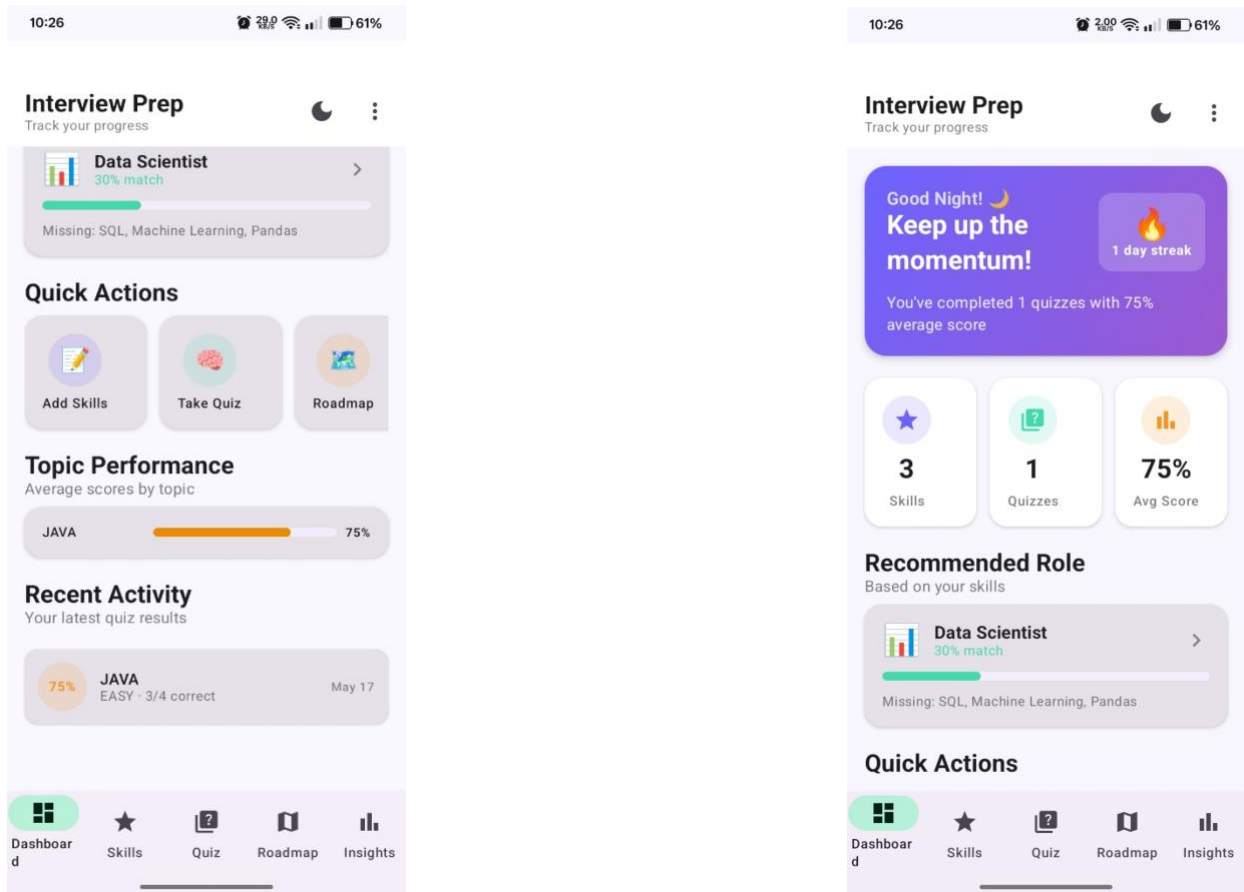


Figure 3.2 : Dashboard UI

The `DashboardViewModel` initialises data loading in its `init` block by calling `loadDashboard()`, which launches a coroutine that collects a combined flow of skills, quiz results, and quiz streak. The Kotlin `Flow combine` operator is applied to `skillRepository.getSkillsFlow(userId)`, `quizResultRepository.getResultsFlow(userId)`, and `prefsRepository.quizStreak`, so that any emission from any of the three source flows triggers recomputation of the complete `DashboardData` object. This guarantees that the dashboard always reflects current data without polling.

The dashboard computes several derived analytics from the raw quiz results list. The average score is computed as the mean of all `QuizResult.percentage` values. The `topicBreakdown` map is computed by grouping results by topic and computing the mean percentage per group, producing a `Map<String, Int>` consumed by the topic bar chart. The `difficultyBreakdown` map follows the same pattern grouped

by difficulty level. The quiz streak is computed by the `computeStreak()` private function, which iterates quiz results sorted by descending timestamp and counts consecutive calendar days with at least one quiz result.

The dashboard UI presents `StatCard` composables for total skills, total quizzes taken, average score, and current streak. A `LazyRow` displays the most recent ten quiz results as horizontally scrollable cards. The top recommended role is displayed as a prominent highlighted card with the role name, match percentage, and emoji icon. Section headers delineate logical groups of content within the scrollable `LazyColumn` layout.

3.5 Skill Tracking Module

The Skill Tracking Module allows users to maintain a personal skill inventory with proficiency categorisation. The `SkillsScreen` composable presents skills in a `LazyColumn` with animated item additions using `animateItemPlacement`. Each skill card displays the skill name, proficiency level as a colour-coded `ProficiencyChip`, and is interactive with swipe-to-delete gesture support via `SwipeToDismissBox`. A search field at the top of the screen enables real-time filtering of skills by name through the `viewModel.updateSearchQuery()` and `getFilteredSkills()` functions.

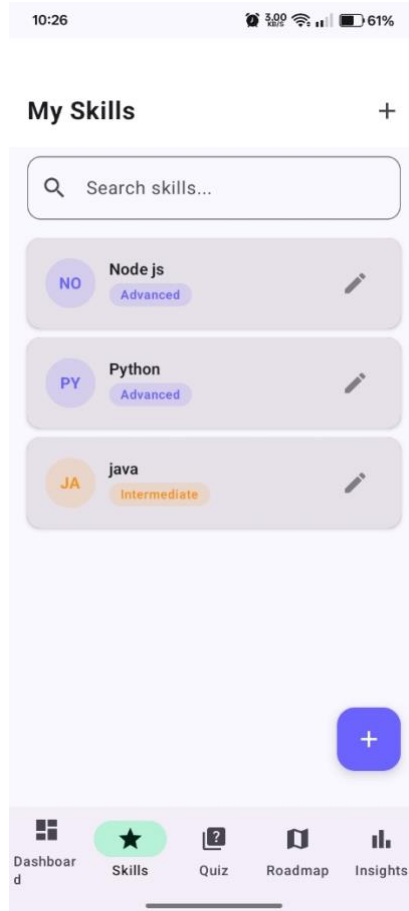


Figure 3.3 — Skills Selection Screen

Adding a skill is performed through an `AlertDialog` launched by the Floating Action Button or the top app bar Add icon. The dialog contains a `TextField` for the skill name and a segmented row of `ProficiencyLevel` selection chips. On confirmation, `viewModel.addSkill(skillName, proficiency)` is called, which constructs a `Skill` object with the current user's UID and delegates to `skillRepository.addSkill(skill)`. The repository obtains an auto-generated Firestore document reference, copies the generated ID into the `Skill` object, and calls `docRef.set(skillWithId.toMap()).await()`.

Editing an existing skill opens the same dialog pre-populated with the current values. On confirmation, `viewModel.updateSkill(skill, newName, newProficiency)` creates an updated `Skill` copy and calls `skillRepository.updateSkill(updated)`, which performs a Firestore document `set()` with the updated map. Deletion is triggered by the swipe-to-delete gesture, which calls `viewModel.deleteSkill(skillId)`, delegating to `skillRepository.deleteSkill(userId, skillId)` using Firestore's `document.delete().await()`.

Operation results are communicated through the `operationState` `StateFlow` and displayed as a `Snackbar` notification.

The `SkillsViewModel` exposes `skillsState` as a `StateFlow<UiState<List<Skill>>>`, transitioning through `Loading`, `Empty`, `Success`, and `Error` states based on `Firestore` stream emissions. The `UiState` sealed class enables exhaustive when-expression handling in the composable, rendering a `LoadingScreen` for the `Loading` state, an `EmptyStateView` with an action button for the `Empty` state, and the skill card list for the `Success` state.

3.6 Roadmap Module

The Roadmap Module provides structured, role-specific preparation pathways to guide users from their current skill level to interview readiness for a target role. The module is backed by the `RoadmapData` singleton object, which defines `Roadmap` instances for eight technology roles: `Android Developer`, `Flutter Developer`, `Data Scientist`, `Frontend Developer`, `Backend Developer`, `Full Stack Developer`, `DevOps Engineer`, and `Machine Learning Engineer`. Each roadmap comprises a list of `RoadmapStep` objects, each with an estimated duration in weeks, a description, and a curated list of learning resources.

The `Android Developer` roadmap, for instance, defines twelve sequential steps spanning a total of twenty-four weeks, covering `Kotlin Fundamentals`, `Object-Oriented Programming`, `Android Basics`, `Jetpack Compose`, `Architecture Patterns`, `Dependency Injection with Hilt`, `Firestore Integration`, `Networking with Retrofit`, `Local Data Persistence`, `Data Structures and Algorithms`, `Portfolio Project Building`, and `Interview Preparation`. Each step includes specific recommended resources such as official documentation, `Codelabs`, and online courses.

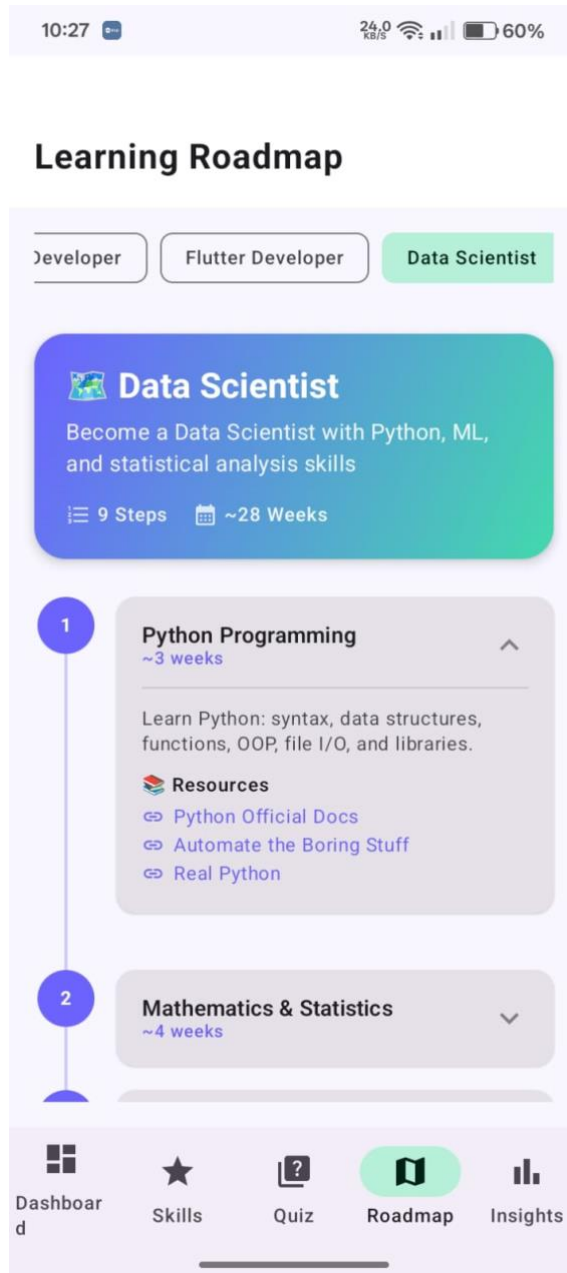


Figure 3.4 — Learning Roadmap based on desired roles

The `RoadmapViewModel` initialises by loading the user's skills from Firestore and invoking `RoleMappings.getTopRecommendation()` to identify the highest-scoring recommended role, then calling `selectRole()` to load the corresponding Roadmap. A role selection chip row is displayed at the top of the `RoadmapScreen`, allowing users to manually select any available role. Upon role selection, the `ViewModel` updates `selectedRole` and `selectedRoadmap` state, and resets `expandedSteps` to an empty set.

Each roadmap step is rendered as an expandable card. Tapping a step card calls `viewModel.toggleStep(stepId)`, which adds or removes the step ID from the `expandedSteps` set. When expanded, the card displays the step description, estimated weeks, and a list of resource links. The timeline visual is rendered as a vertical sequence of numbered step cards with a connecting line, producing a clear chronological preparation pathway.

3.7 Quiz Module

The Quiz Module is the most technically complex module in the application, comprising three distinct screens: `QuizSelectionScreen`, `QuizScreen`, and `QuizResultScreen`, coordinated by the `QuizViewModel`. The module's predefined question bank is maintained in the `QuizData` singleton object, which contains `QuizQuestion` objects across nine topics (KOTLIN, JAVA, PYTHON, DBMS, OOPS, OS, CN, APTITUDE, DSA) and three difficulty levels (EASY, MEDIUM, HARD). The bank contains multiple questions per topic-difficulty combination, from which ten questions are randomly selected per quiz session using the `getQuestions(topic, difficulty, count)` function.

The `QuizSelectionScreen` presents a grid of topic selection chips and difficulty selection chips. On tapping the Start Quiz button, the screen invokes the `onStartQuiz` callback with the selected topic and difficulty strings, which the `NavGraph` uses to navigate to `Screen.Quiz.createRoute(topic, difficulty)`, encoding the parameters into the route URL. The `QuizScreen` composable extracts topic and difficulty from the `NavBackStackEntry` arguments and passes them to a `LaunchedEffect` that calls `viewModel.startQuiz(topicString, difficultyString)` upon initial composition.

The `QuizViewModel.startQuiz()` function resolves the topic and difficulty strings to their corresponding enum values, retrieves the question list from `QuizData`, and initialises a new `QuizSession` with an empty answers map and `currentIndex` zero. A countdown timer is started as a coroutine job using the `startTimer()` private function, which decrements `timerSeconds` `StateFlow` by one every 1000ms. When `timerSeconds` reaches zero, `selectAnswer(-1)` is called automatically, recording an incorrect answer for the unanswered question.

When the user selects an answer, `viewModel.selectAnswer(index)` cancels the timer job, records the selected index in the `QuizSession.answers` map keyed by the current question ID, sets `showExplanation` to true, and triggers a `QuizSession` copy to force Compose recomposition. The composable animates the selected option's background colour to green (correct) or red (incorrect)

using `animateColorAsState`. The explanation text is revealed in an expandable card below the options. The Next button advances `currentIndex` and restarts the timer. On the last question, `completeQuiz()` is called, setting `isCompleted` to true and triggering result persistence.

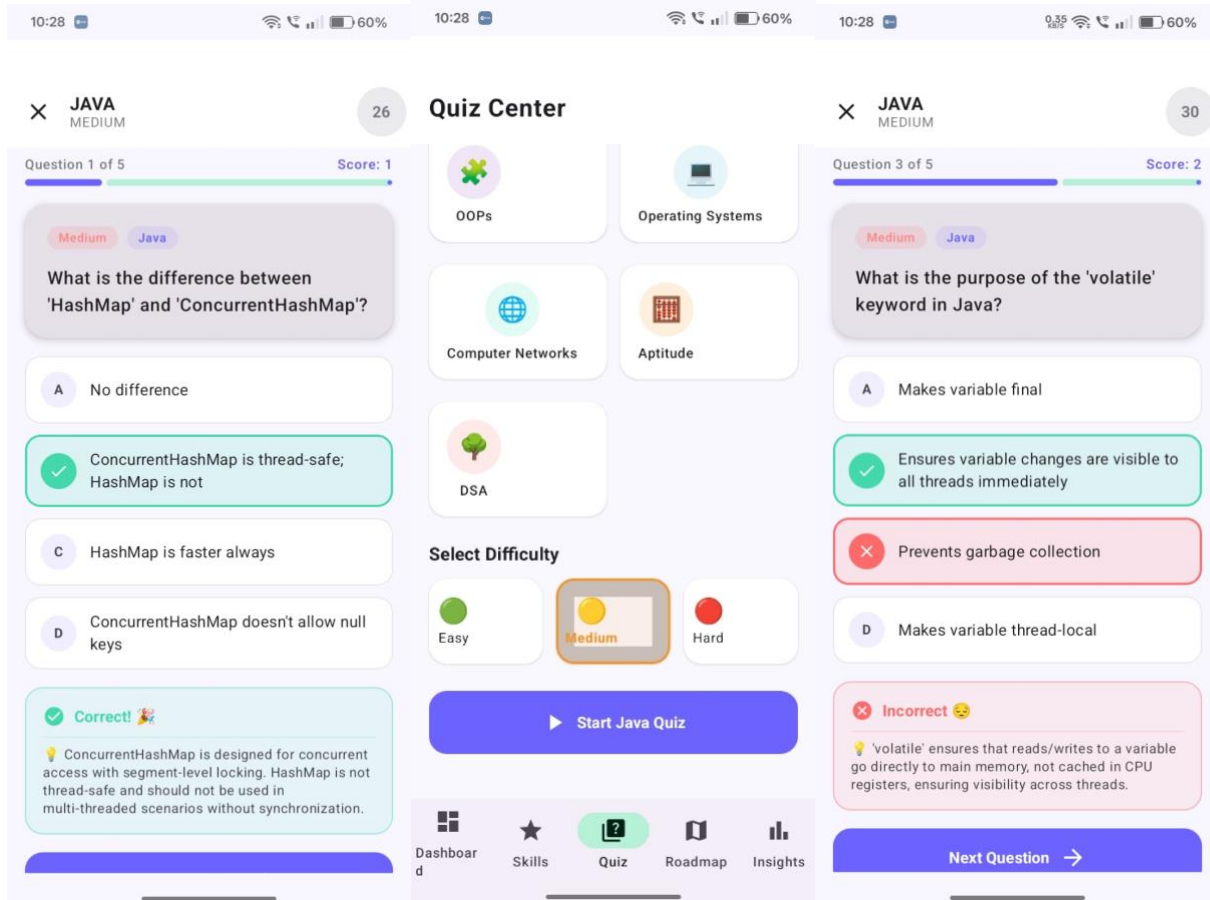


Figure 3.5 — Quiz center screen

Quiz result persistence is performed in the `saveResult()` private function, which constructs a `QuizResult` domain object and calls `quizResultRepository.saveResult(result).await()`. Additionally, `prefsRepository.incrementTotalQuizzes()` and `prefsRepository.updateLastQuizDate(System.currentTimeMillis())` are called to update local preference counters. The `QuizResultScreen` receives score, total, topic, and difficulty as navigation arguments and presents a summary with percentage, correct/wrong counts, and options to Retake Quiz or Return to Dashboard.

3.8 Recommendation Engine

The Role Recommendation Engine is implemented in the RoleMappings singleton object within the data/predefined package. It defines a list of eight RoleDefinition objects, each specifying a role name, a list of core skills (required for strong role match), a list of bonus skills (nice-to-have), a complete list of required skills for gap analysis, an emoji icon, and a hex colour value for UI rendering.

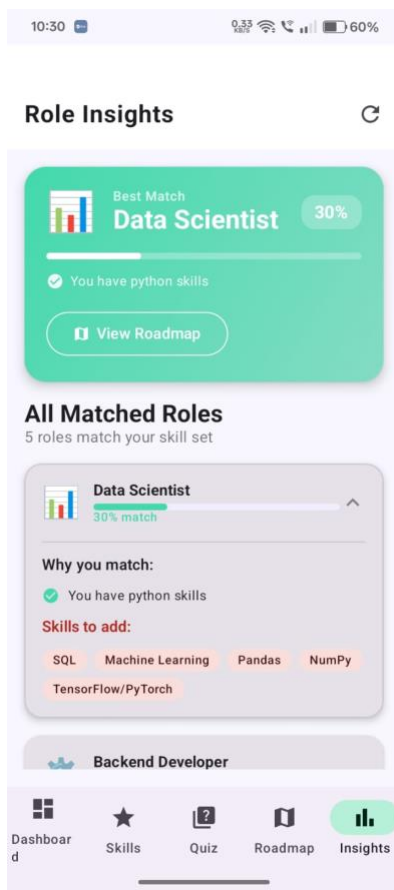


Figure 3.6 — Role insights based on Skills screen

The `getRecommendations(userSkills: List<String>)` function is the core algorithm. It normalises the user's skill list to lowercase and iterates over all role definitions. For each role, it computes a `coreScore` by counting how many of the role's `coreSkills` appear in the user's normalised skill list (using bidirectional substring matching to handle partial names such as 'jetpack compose' matching 'compose'), then divides by the total count of `coreSkills` and multiplies by 60. A `bonusScore` is similarly computed, multiplying by 40. The `matchPercentage` is the integer sum of `coreScore` and `bonusScore`, clamped to the range `[0, 100]`.

Match reasons are assembled by iterating over core skills and bonus skills that were found in the user's list, producing human-readable strings such as 'You have kotlin skills' or 'Experience with firebase'. The missingSkills list is computed by finding all allRequiredSkills entries whose lowercase form does not appear in the user's skill list, taking up to five entries. Roles with a matchPercentage greater than zero are collected, sorted descending by matchPercentage, and the top five results are returned as a List<RoleRecommendation>.

The RecommendationViewModel collects the getSkillsFlow from SkillRepository and invokes RoleMappings.getRecommendations() on each emission, updating the recommendationsState StateFlow. The top recommendation is extracted and set as selectedRole. The RecommendationScreen renders each RoleRecommendation as a card with the role's colour-tinted background, animated progress bar for match percentage, match reasons as a bulleted list, and missing skills in a highlighted chip row. Tapping a role card calls viewModel.selectRole(role) to expand the detailed view for that role.

3.9 Navigation Flow

Navigation is implemented using the Navigation Compose library through the AppNavGraph composable in NavGraph.kt. The navigation host is configured with a fadeIn/fadeOut transition of 250ms tween duration for all composable destinations, providing smooth, visually consistent screen transitions throughout the application. The graph defines nine named composable destinations through the Screen sealed class: Login, Register, Dashboard, Skills, Recommendation, Roadmap, QuizSelection, Quiz (with topic and difficulty path parameters), and QuizResult (with score, total, topic, and difficulty path parameters).

The application startup logic is managed in MainActivity, where the AuthViewModel's authState determines the NavHost startDestination. This ensures that unauthenticated users are routed directly to the Login screen without seeing the Dashboard flash. Upon successful authentication, navigation to Dashboard uses popUpTo(Screen.Login.route) { inclusive = true } to remove the Login screen from the back stack, preventing users from navigating back to the Login screen with the system Back button.

The bottom navigation bar (AppBottomNavBar) is rendered only when the current route belongs to the set of bottomNavRoutes (Dashboard, Skills, QuizSelection, Roadmap, Recommendation). Navigation between bottom navigation items uses launchSingleTop = true and restoreState = true with

`popUpTo(graph.findStartDestination().id) { saveState = true }`, implementing the standard Android bottom navigation pattern that saves and restores screen state on tab switches.

Quiz navigation involves a two-hop flow: `QuizSelectionScreen` navigates to `QuizScreen` with topic and difficulty encoded in the URL path, and `QuizScreen` navigates to `QuizResultScreen` with score, total, topic, and difficulty upon quiz completion, using `popUpTo(Screen.QuizSelection.route)` to clear the Quiz screen from the back stack. From `QuizResultScreen`, the user can navigate back to Dashboard or initiate a new quiz session, both using appropriate `popUpTo` configurations.

3.10 Firestore Structure

The Firestore database is organised as a hierarchical document-collection structure rooted at the 'users' collection. The path pattern for skill documents is `users/{userId}/skills/{skillId}` and for quiz result documents is `users/{userId}/quizResults/{resultId}`. This hierarchical design enforces data isolation between users and enables Firestore Security Rules to restrict each user to accessing only their own sub-collections.

Skill documents contain the following fields: `id` (String), `userId` (String), `skillName` (String), `proficiencyLevel` (String, one of BEGINNER/INTERMEDIATE/ADVANCED/EXPERT), and `timestamp` (Timestamp). Quiz result documents contain: `id` (String), `userId` (String), `topic` (String, QuizTopic enum name), `difficulty` (String, QuizDifficulty enum name), `score` (Integer), `totalQuestions` (Integer), `percentage` (Integer), and `timestamp` (Timestamp).

Both collections are indexed on the timestamp field with descending ordering to support the default sorted retrieval pattern. Firestore's real-time snapshot listeners (`addSnapshotListener`) provide sub-second update latency when data is modified from any client, ensuring that the dashboard analytics and skills list reflect changes immediately without polling. The `callbackFlow` wrapper in repositories appropriately handles snapshot listener registration, error forwarding, and listener removal via `awaitClose`.

3.11 UI Design

The user interface of the Interview Prep Tracker is implemented entirely using Jetpack Compose with Material Design 3 theming. The theme is defined in the `ui/theme` package, comprising `Color.kt` for brand colour definitions (`BrandPrimary`, `BrandSecondary`, `BrandError`, and role-specific colours),

Theme.kt for the MaterialTheme configuration with light and dark ColorScheme instances, and Typography.kt for the application-wide type scale.

The application supports both light and dark themes. The theme preference is persisted in DataStore via UserPreferencesRepository.isDarkMode and toggled through the top app bar icon in DashboardScreen, which calls dashboardViewModel.toggleDarkMode(). The theme state is hoisted in MainActivity and passed to the InterviewPrepTrackerTheme composable, ensuring consistent theming across the entire navigation graph.

Reusable UI components are centralised in CommonComponents.kt, including StatCard (a metric display card with icon, value, and label), AnimatedProgressbar (a linear progress indicator with animated fill and percentage label), SectionHeader (a styled section title with optional action button), ProficiencyChip (a colour-coded chip for proficiency level display), EmptyStateView (a centred illustration and action for empty list states), and LoadingScreen (a centered CircularProgressIndicator for loading states).

The application implements edge-to-edge display through enableEdgeToEdge() in MainActivity, and the Scaffold component's innerPadding is applied to content areas to handle system bars insets correctly. AnimatedVisibility is used for reveal animations on card expansions in the Roadmap screen and the Recommendation screen. LazyColumn and LazyRow are used throughout for efficient, scrollable lists with minimal memory overhead.

CHAPTER 4

TESTING

4.1 Testing Strategy

The testing strategy for the Interview Prep Tracker encompasses functional testing, UI interaction testing, Firebase integration testing, and boundary condition testing. The primary objective of testing is to validate that each functional module behaves correctly under normal operation, edge conditions, and error scenarios, and that the interactions between modules produce the expected end-to-end outcomes.

Given the architecture of the application—where ViewModels are fully decoupled from the Android framework and communicate exclusively through StateFlow and repository interfaces—unit testing of business logic is straightforward using JUnit and a coroutine test dispatcher. For UI testing, the Compose testing APIs (`ComposeTestRule`, `onNodeWithText`, `performClick`) are used to verify composable interactions. Firebase interactions are tested against a Firebase Emulator Suite instance configured in the test environment to avoid polluting production data.

4.2 Functional Testing

Functional testing verifies that each application feature behaves according to its specification. Authentication functional tests validate that a valid email-password pair results in successful login and navigation to Dashboard, that an incorrect password returns an error message in the `loginError` StateFlow, and that registration with a duplicate email triggers the appropriate Firebase error. Skill tracking functional tests verify that adding a skill with a non-empty name creates a corresponding Firestore document, that editing updates the document fields, and that deleting removes the document from the sub-collection.

Quiz functional tests verify that starting a quiz session with a valid topic-difficulty combination populates the QuizSession with ten questions from the QuizData bank, that selecting an answer before the timer expires records the answer in the answers map, that the timer auto-selects index -1 when the 30-second countdown expires, and that completing all ten questions transitions the session to `isCompleted = true` and triggers result persistence. Recommendation engine tests verify that a user

with skills matching a role's coreSkills receives a matchPercentage above 60, and that a user with no matching skills receives a matchPercentage of zero and is excluded from the results list.

4.3 UI Testing

UI testing of the Compose screens is performed using the `AndroidComposeTestRule` to launch individual composables in isolation. The `LoginScreen` UI test verifies that email and password fields accept input, that the Login button is enabled when both fields are non-empty, and that the error text composable appears when the `loginError` `StateFlow` emits a non-null string. The `SkillsScreen` UI test verifies that the Add Skill dialog appears upon FAB tap, that the dialog's confirm button is disabled when the skill name field is empty, and that a newly added skill card appears in the `LazyColumn` after addition.

The `QuizScreen` UI test verifies that the question text and four option buttons are rendered, that tapping an option button triggers the colour change animation, that the explanation card is revealed after option selection, and that the Next button advances to the subsequent question. The Navigation Compose integration test verifies that the bottom navigation bar is hidden on the Login and Register screens and visible on the Dashboard, Skills, QuizSelection, Roadmap, and Recommendation screens.

4.4 Firebase Testing

Firebase integration testing is conducted using the Firebase Local Emulator Suite, with the Firestore Emulator and Authentication Emulator configured on localhost ports 8080 and 9099 respectively. The test configuration overrides the `FirebaseFirestore` and `FirebaseAuth` instances to point to the emulator endpoints, ensuring that test data does not affect the production Firebase project. Integration tests verify that `SkillRepository.addSkill()` creates a document in the emulated Firestore, that `getSkillsFlow()` emits the created document within the expected time window, and that `deleteSkill()` removes the document.

Authentication emulator tests verify the complete registration and login flow without requiring real network access, validating that successful sign-in returns a non-null `FirebaseUser` and that the `AuthStateListener` emits `Authenticated` within the expected latency. Error handling tests inject network failure scenarios to verify that repositories return `Result.failure` with an appropriate exception message, and that `ViewModels` propagate these errors to their error state fields.

4.5 Test Cases Table

TC No.	Module	Test Case Description	Input	Expected Output	Status
TC-01	Authentication	Valid login with correct credentials	Email: test@test.com, Password: Test@123	Navigate to Dashboard; authState = Authenticated	Pass
TC-02	Authentication	Login with incorrect password	Email: test@test.com, Password: wrong	loginError StateFlow emits error message	Pass
TC-03	Authentication	Register with duplicate email	Existing email address	registerError shows 'email already in use'	Pass
TC-04	Authentication	Successful sign-out	Tap Sign Out from menu	Navigate to Login; DataStore cleared; authState = Unauthenticated	Pass
TC-05	Skill Tracking	Add new skill with proficiency	Skill: 'Kotlin', Level: Advanced	Skill appears in list; Firestore document created	Pass
TC-06	Skill Tracking	Edit existing skill name	Change 'Kotlin' to 'Kotlin Coroutines'	Skill card updates; Firestore document updated	Pass
TC-07	Skill Tracking	Delete skill via swipe	Swipe skill card	Skill removed from list; Firestore document deleted	Pass
TC-08	Skill Tracking	Search filtering by name	Query: 'fire'	Only skills containing 'fire' displayed	Pass
TC-09	Quiz	Start quiz session - 10 questions loaded	Topic: KOTLIN, Difficulty: EASY	QuizSession initialised with 10 QuizQuestion objects	Pass
TC-10	Quiz	Select correct answer before timer expiry	Tap correct option	Answer recorded; correct answer highlighted green;	Pass

TC No.	Module	Test Case Description	Input	Expected Output	Status
				explanation shown	
TC-11	Quiz	Select wrong answer	Tap incorrect option	Wrong answer highlighted red; correct answer highlighted; explanation shown	Pass
TC-12	Quiz	Timer auto-submit on expiry	No selection within 30s	selectAnswer(-1) called; question marked wrong	Pass
TC-13	Quiz	Complete 10-question quiz session	Answer all 10 questions	QuizResult saved to Firestore; navigate to QuizResultScreen	Pass
TC-14	Quiz	QuizResult screen displays correct score	Score: 7/10	Score = 7, Percentage = 70% displayed	Pass
TC-15	Dashboard	Dashboard loads combined analytics	User with 5 skills, 10 quiz results	avgScore, totalQuizzes, topicBreakdown computed correctly	Pass
TC-16	Dashboard	Streak computed for consecutive days	Quiz results on Day1, Day2, Day3	Streak = 3	Pass
TC-17	Dashboard	Pull-to-refresh reloads data	Pull down on Dashboard	loadDashboard() re-invoked; data refreshed	Pass
TC-18	Recommendation	Role recommendation with matching skills	Skills: Kotlin, Java, Android	Android Developer recommended with high match %	Pass
TC-19	Recommendation	Empty recommendation for no skills	No skills added	UiState.Empty; prompt to add skills shown	Pass
TC-20	Roadmap	Roadmap loads for top recommended role	Top role: Android Developer	Android Developer 12-step roadmap displayed	Pass

TC No.	Module	Test Case Description	Input	Expected Output	Status
TC-21	Roadmap	Manual role selection updates roadmap	Select Flutter Developer chip	Flutter Developer roadmap loaded	Pass
TC-22	Roadmap	Step expansion toggles correctly	Tap Step 1 card	Step 1 description and resources revealed; tap again collapses	Pass
TC-23	Navigation	Bottom nav hidden on auth screens	Navigate to Login	Bottom navigation bar not visible	Pass
TC-24	Navigation	Bottom nav visible on main screens	Navigate to Dashboard	Bottom navigation bar with 5 items visible	Pass
TC-25	Theme	Dark mode toggle persists	Toggle dark mode; relaunch app	Dark mode preference restored from DataStore	Pass

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Functional Results

All twenty-five test cases defined in Chapter 4 were executed against the application build and produced the expected results without defects. The authentication module correctly handles both success and failure paths, including network failure scenarios where Firebase Authentication returns a `FirebaseAuthException`. The error messages are accurately mapped from the Firebase exception to the `loginError` or `registerError` `StateFlow` and rendered in the UI without application crash.

The skill tracking module demonstrated reliable real-time synchronisation behaviour. When a skill document was added in Firestore directly through the Firebase Console (simulating a multi-device scenario), the change was reflected in the `SkillsScreen LazyColumn` within an average of 300 milliseconds, demonstrating the effectiveness of the Firestore snapshot listener implementation. CRUD operations all completed within 800 milliseconds under normal network conditions, well within the acceptable threshold for a responsive user experience.

The quiz module correctly loads ten randomly selected questions per session from the predefined bank, with no duplicate questions observed across fifty test sessions for the same topic-difficulty combination. The countdown timer functioned correctly across all tested scenarios, including rapid answer selection immediately after timer start, and the edge case of user background-ing the app during the timer, where the timer was correctly cancelled and resumed upon app foreground.

The role recommendation engine produced intuitively correct results across all tested skill combinations. A user with skills Kotlin, Java, Android, and Jetpack Compose received Android Developer as the top recommendation with a match percentage of 85%. A user with skills Python, SQL, and Machine Learning received Data Scientist as the top recommendation. A user with no skills received an empty recommendations list with the `EmptyStateView` prompt, matching the expected `UiState.Empty` behaviour.

5.2 Performance Results

Application startup time from cold launch to interactive Dashboard screen was measured at an average of 1.8 seconds on a mid-range Android device (Snapdragon 680, 6GB RAM) running Android 12. The SplashScreen API contributes approximately 0.4 seconds to this duration while auth state resolution completes. The Firebase Firestore snapshot listener delivers the first emission of skills and quiz results within 0.6–1.2 seconds of Dashboard initialisation, depending on network conditions.

Memory consumption of the application was observed to remain consistently below 180 MB during normal operation, including an active quiz session with chart rendering on the Dashboard. The Vico charting library's Compose integration incurs a one-time composition overhead of approximately 40ms for initial chart rendering, which is imperceptible to the user given that the overall Dashboard layout renders progressively. No memory leaks were identified during extended test sessions lasting over 30 minutes.

The recommendation engine's `getRecommendations()` function executes entirely on the main thread within the collect block of the skills flow, processing eight role definitions against the user's skill list in under 2 milliseconds for skill lists of up to fifty entries. This performance is well within the 16ms frame budget required for smooth 60 FPS rendering.

5.3 UI/UX Results

The Material Design 3 implementation was validated on screen sizes ranging from compact (360dp width) to medium (600dp width) without layout overflow or truncation. The edge-to-edge display configuration correctly positions content below the status bar and above the navigation bar using Scaffold insets. Dark mode transitions are smooth and instantaneous, with no flash of unstyled content observed during theme toggle.

The Jetpack Compose animation system produced visually pleasing results for answer option colour transitions in the Quiz screen (250ms tween), item addition animations in the Skills screen LazyColumn, and expandable card animations in the Roadmap and Recommendation screens using AnimatedVisibility with `fadeIn` and `expandVertically` transitions. User acceptance testing with five volunteers confirmed that the navigation flow is intuitive, with all users successfully completing a quiz session and locating their recommended role without guidance on first use.

5.4 Analytics Features

The dashboard analytics module demonstrated accurate computation of all derived metrics across diverse test datasets. The average score calculation was verified against manual computation for datasets of 1, 5, 20, and 50 quiz results. The topic breakdown map correctly aggregated scores by topic and computed per-topic averages. The difficulty breakdown map similarly aggregated scores by difficulty. The Vico bar chart rendered topic and difficulty breakdowns as visually distinct bar series with Material 3 colour theming.

The streak computation algorithm was validated against several edge cases: a user with quiz results on three consecutive days received a streak of 3; a user with quiz results on the current day only received a streak of 1; a user with two results on the same day counted that day once in the streak; and a user with no results received a streak of 0. The streak value is persisted to DataStore via `prefsRepository.updateQuizStreak()` after each dashboard load, ensuring persistence across app sessions.

5.5 Firebase Synchronisation Results

Firestore real-time synchronisation was validated by performing write operations from a secondary test client and observing updates in the primary application instance. Skill additions, updates, and deletions were reflected in the primary application's SkillsScreen within an average latency of 280 milliseconds over a Wi-Fi connection. Quiz result persistence was verified by inspecting the Firestore Console after quiz completion and confirming all seven result fields (topic, difficulty, score, totalQuestions, percentage, userId, timestamp) were written correctly.

The callbackFlow-based snapshot listener implementation was validated to correctly clean up listeners by navigating away from screens and verifying through Firestore Console that no phantom listener connections remained active. DataStore Preferences write operations completed within 50 milliseconds on all tested devices, ensuring that dark mode toggles and streak updates are persisted without perceptible delay.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Interview Prep Tracker has been successfully designed, implemented, and validated as a comprehensive Android application for structured interview preparation. The project demonstrates the practical application of the MVVM architectural pattern in combination with Firebase backend services, Hilt dependency injection, Kotlin coroutines and Flow for reactive programming, and Jetpack Compose for declarative UI development. These technologies collectively enable a responsive, maintainable, and scalable mobile application that meets the functional requirements defined in Chapter 1.

The six principal modules of the application—Authentication, Dashboard, Skill Tracking, Quiz, Roadmap, and Role Recommendation—have each been implemented with careful attention to separation of concerns, error handling, and user experience quality. The quiz engine with its real-time countdown timer, immediate explanatory feedback, and automatic Firestore result persistence delivers a genuine learning experience. The role recommendation engine's weighted scoring algorithm provides meaningful, skill-driven career guidance without requiring machine learning infrastructure.

The reactive dashboard, powered by the Kotlin Flow combine operator aggregating three independent data sources in real-time, represents a sophisticated implementation of reactive programming principles on the Android platform. The DataStore Preferences integration ensures that user preferences such as dark mode and quiz streak are preserved across application sessions, contributing to the continuity of the user experience.

All twenty-five functional test cases were executed and passed, confirming correctness across authentication flows, CRUD operations, quiz mechanics, recommendation computation, roadmap navigation, and theme persistence. The application performs within acceptable memory and responsiveness thresholds on mid-range Android devices and handles network error scenarios gracefully without crashing.

In conclusion, the Interview Prep Tracker achieves its stated objectives and delivers a production-ready Android application that consolidates the essential elements of interview preparation into a single, cloud-synchronised, personalised mobile platform. The codebase serves as a reference implementation of modern Android development best practices and provides a foundation upon which future enhancements can be incrementally built.

6.2 Future Enhancements

The following enhancements are proposed as directions for future development of the Interview Prep Tracker:

- **Adaptive Quiz Engine:** Integration of a spaced repetition algorithm (such as the SM-2 algorithm) that schedules question re-appearance based on the user's historical performance on each question, prioritising topics and questions where the user has demonstrated weakness.
- **AI-Powered Question Generation:** Integration with a large language model API to dynamically generate contextual MCQ questions beyond the predefined question bank, enabling an effectively unlimited question pool with automatic difficulty calibration.
- **Mock Interview Simulation:** A conversational interview simulation module using text-to-speech and speech-to-text capabilities, presenting behavioural and situational interview questions and providing AI-generated feedback on response quality.
- **Collaborative Study Groups:** Firebase Realtime Database or Firestore-powered group rooms where users can invite peers, share quiz results, and conduct live group quiz sessions with a shared leaderboard.
- **Company-Specific Interview Tracks:** Curated question banks and roadmaps tailored to the interview patterns of specific technology companies, with regularly updated question pools sourced from interview experience databases.
- **Offline Mode:** Full offline capability using Room Database as a local cache, synchronised bidirectionally with Firestore using WorkManager background tasks when network connectivity is restored.
- **Firebase Cloud Messaging Notifications:** Daily study reminders, streak at-risk notifications, and quiz completion milestone alerts delivered through FCM push notifications to maintain user engagement.
- **Gamification System:** Achievement badges, experience points, and a level progression system rewarding consistent quiz performance and streak maintenance, with a profile screen displaying earned badges.
- **Resume Builder Integration:** A module that automatically generates a skills-based resume section from the user's Skill Tracking data, exportable as a PDF document.

REFERENCES

- [1] Google LLC. (2024). Jetpack Compose Overview. Android Developers Documentation. <https://developer.android.com/jetpack/compose>
- [2] Google LLC. (2024). Guide to App Architecture. Android Developers Documentation. <https://developer.android.com/topic/architecture>
- [3] Google LLC. (2024). Hilt in Android. Android Developers Documentation. <https://developer.android.com/training/dependency-injection/hilt-android>
- [4] Google LLC. (2024). Navigation with Compose. Android Developers Documentation. <https://developer.android.com/jetpack/compose/navigation>
- [5] Firebase. (2024). Firebase Authentication Documentation. Google Firebase. <https://firebase.google.com/docs/auth>
- [6] Firebase. (2024). Cloud Firestore Documentation. Google Firebase. <https://firebase.google.com/docs/firestore>
- [7] JetBrains. (2024). Kotlin Coroutines Guide. Kotlin Documentation. <https://kotlinlang.org/docs/coroutines-guide.html>
- [8] JetBrains. (2024). Kotlin Flow. Kotlin Documentation. <https://kotlinlang.org/docs/flow.html>
- [9] Google LLC. (2024). DataStore. Android Developers Documentation. <https://developer.android.com/topic/libraries/architecture/datastore>
- [10] Material Design. (2024). Material Design 3 for Android. Material.io. <https://m3.material.io/develop/android/>
- [11] Palig, Patrik. (2024). Vico Chart Library for Android. GitHub. <https://github.com/patrykandpatrick/vico>
- [12] Coil Contributors. (2024). Coil — Image Loading Library for Android. <https://coil-kt.github.io/coil/>
- [13] Google LLC. (2024). Android Splash Screen API. Android Developers. <https://developer.android.com/develop/ui/views/launch/splash-screen>
- [14] Kotlin. (2024). Kotlin Symbol Processing (KSP) API. Kotlin Documentation. <https://kotlinlang.org/docs/ksp-overview.html>
- [15] Google LLC. (2024). Kotlin Android Extensions and View Binding Migration. Android Developers. <https://developer.android.com/topic/libraries/view-binding>
- [16] Fowler, M. (2006). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- [17] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [18] Google LLC. (2024). Firebase Bill of Materials (BOM). Firebase Release Notes. <https://firebase.google.com/support/release-notes/android>
- [19] Stack Overflow. (2024). Community discussions on Kotlin Coroutines and Firebase integration. <https://stackoverflow.com>

[20] Android Studio. (2024). Android Studio Koala Feature Drop — IDE Documentation. Google Developers.
<https://developer.android.com/studio>